

UCT CLOUD / DEVOPS BOOTCAMP

Deploying a Static Website on Amazon S3

From an AI-generated website to a live, public AWS endpoint

Presented by Sylvain “Sims” Kamdem

IT Infrastructure Engineer • AWS Certified

AGENDA

What we'll cover

01

Build the website with AI

Prompting an AI assistant to generate a clean, ready-to-host HTML/CSS/JS site

02

Why Amazon S3 for hosting

Object storage as a fast, low-cost home for static sites

03

Step-by-step deployment

Bucket creation, hosting config, upload, and public access

04

Going further

Custom domains, HTTPS, and CloudFront in front of S3

05

Troubleshooting & takeaways

Common errors and the key ideas to remember

PART 1

Build the Website with AI

Before we touch AWS, let's generate the site we're going to host.

Why start with an AI-built website?



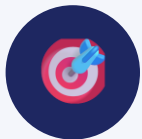
Speed

A working, styled HTML/CSS/JS site in minutes instead of hours of boilerplate



No blank page

AI drafts structure, copy, and layout — you refine instead of starting from zero



Focus shifts

Your effort goes into prompting well and testing on AWS, not hand-coding tags



One clean file

A single self-contained HTML file is exactly what S3 wants to serve

How to prompt the AI effectively

1

Be specific about purpose

Say who the site is for, its goal, and the sections you need (hero, about, services, contact)

2

Ask for one self-contained file

Request inline CSS and JS in a single index.html — simplest to upload and host on S3

3

Set the tone and style

Describe the look: colors, mood, fonts, professional vs. playful

4

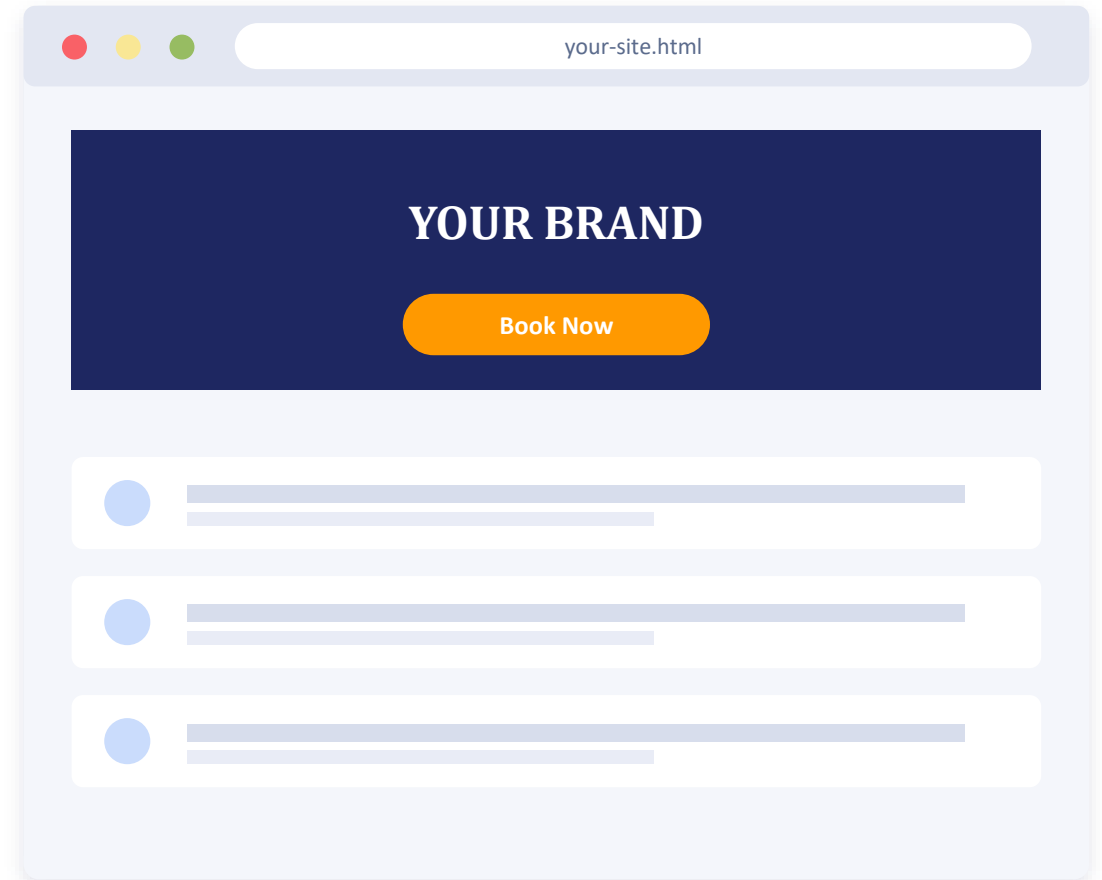
Iterate in small steps

Refine copy, spacing, and colors one request at a time rather than rewriting everything

Example prompt → ready-to-host site

PROMPT

“Create a single self-contained index.html for a DJ entertainment business. Include a hero section with a call-to-action, an About section, a Services grid, and a Contact section. Use a dark, modern look with one bold accent color. Inline all CSS and JS — no external files.”



PART 2

Hosting on Amazon S3

Turning that HTML file into a public, live website.

What is S3 static website hosting?

Amazon S3 (Simple Storage Service) is object storage — but it can also serve HTML, CSS, JS, and image files directly over HTTP, no web server required.



No servers to manage

Fully managed storage — no EC2 instance or OS to patch



Highly durable & available

Built for 99.999999999% durability of objects



Pay for what you use

Cents per GB stored plus small request/transfer costs



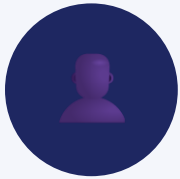
Great for the right use case

Portfolios, landing pages, docs sites, single-page apps



*An S3 bucket can act as
a simple, public web host.*

Before we deploy: prerequisites



An AWS account

With permissions to create and configure S3 buckets



Your website files

At minimum an index.html, plus an error.html and any assets



Comfort with public access

Understanding that hosting publicly means the objects are world-readable

Steps 1–2: Bucket & hosting config

1

Create the S3 bucket

- Go to S3 → Create bucket
- Choose a globally unique bucket name
- Pick the AWS Region closest to your users

2

Enable static website hosting

- Open the bucket → Properties tab
- Static website hosting → Enable
- Set index.html as the index document
- Set error.html as the error document

Steps 3–4: Upload & open access

3

Upload your website files

- Objects tab → Upload
- Add index.html, error.html, and assets
- Or use the AWS CLI: `aws s3 cp / sync`

4

Allow public read access

- Permissions tab → turn off “Block all public access”
- Add a bucket policy granting `s3:GetObject` to everyone
- Save — objects are now publicly readable

⚠ Public read access is intentional for a public website — never store secrets or private data in this bucket.

Step 5: test it — and go further

5

Copy the bucket website endpoint and open it in a browser

Properties → Static website hosting → Bucket website endpoint
Format: `http://bucket-name.s3-website-region.amazonaws.com`

Optional next steps for a production-grade setup:

CloudFront

CDN in front of S3 for caching, speed, and HTTPS

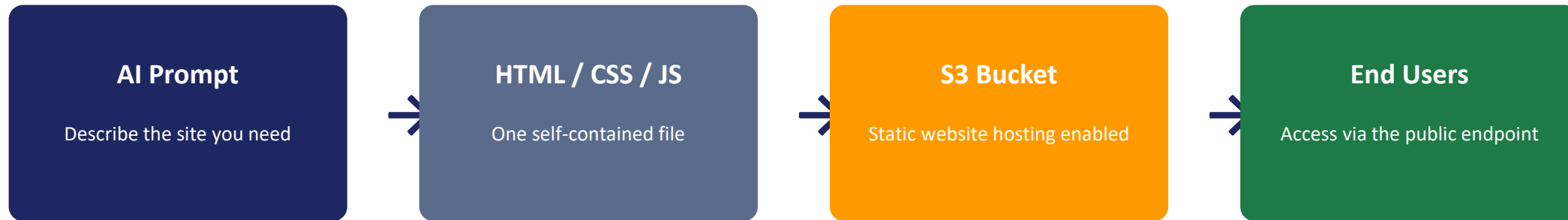
Route 53

Point a custom domain name at your site

ACM Certificate

Free SSL/TLS certificate for HTTPS on your domain

End-to-end flow



A prompt becomes a file, the file becomes a bucket object, and the bucket becomes a public website.

WRAPPING UP

Common issues & how to fix them

403 Forbidden

Public access is blocked or the bucket policy is missing/incorrect

Recheck “Block public access” settings and the s3:GetObject policy

404 on refresh

Error document isn’t set, or the wrong file name was uploaded

Confirm index.html / error.html names in hosting settings

Site not updating

Browser or CDN cache is serving an old version

Hard-refresh, or invalidate the CloudFront cache

Mixed content warning

Site loaded over HTTP has HTTPS-only resources, or vice versa

Use CloudFront + ACM certificate to serve everything over HTTPS

WRAPPING UP

Key takeaways



AI can generate a clean, self-contained website in minutes — good prompting matters more than hand-coding



S3 turns that HTML file into a public website with no servers to manage



Hosting is five steps: create bucket → enable hosting → upload → open access → test the endpoint



CloudFront, Route 53, and ACM take the same bucket to a custom domain with HTTPS

THANK YOU

Questions?

Sylvain “Sims” Kamdem

github.com/sylvain-skamdem • linkedin.com/in/sylvainsimo